University College of Applied Sciences.
Computer Engineering Department.
Data Science and Artificial Intelligence.

Smart Text Analyzer

Project Report

Student Name: **Amal Ahmed Abu Elba**.
Student ID: **220231850**.
Course: **Data Structure.**
Instructor: **Eng.Abd-Elkarem Abu Samra.**
Date: 18th-Jun-2025.

1. Introduction

The Smart Text Analyzer is a command-line Python application designed to assist users in analyzing and extracting insights from textual data. It provides features such as smart autocompletion, next-word prediction, sentence-level sentiment analysis, keyword extraction, and word cloud data preparation, word/character statistics, word search and replacement(with undo). The goal is to deliver a simple yet powerful tool that demonstrates the effective use of data structures, natural language processing techniques, and user interaction principles.

2. System Design

2.1 Overall Architecture

The program uses modular function-based design, enhanced with object-oriented principles (OOP). It consist of:

- ✓ Main Menu Loop: Displays options and routes the user to the corresponding function.
- ✓ A class SmartTextAnalyzer to manage state(text_data, word_list, sentence_list, etc..).
- ✓ Text Loading & Preprocessing: Allows the user to input text manually or from a file and prepares the text by lowercasing and removing punctuation.
- ✓ An undo stack to support rollback of recent replacement.
- ✓ Functional Modules: Each feature is implemented as a separate function.

2.2 Data Structures Used

1. Trie for Autocompletion:

- Why: Efficient for prefix-based search and suggestion.
- Advantages: Fast insertions and prefix lookups (O(n), where n is prefix length).
- Alternative: Lists (less efficient, require scanning all entries).

2. Dictionary for Bigram Model (Next Word Prediction):

- Why: Captures word transitions and frequencies efficiently.
- Advantages: O(1) average time complexity for updating and accessing word pairs.
- Alternative: Lists or nested lists (less readable, slower lookup).

3. Sets for Sentiment Lexicons:

- Why: Allows constant time membership tests.
- Advantages: Clean handling of positive/negative words.
- Alternative: Lists (slower in large-scale membership checking).

4. Dictionary for Keyword and Word Cloud Frequencies:

- Why: Efficient frequency counting and normalization.
- Advantages: Direct access to word frequency and scalable.

5. Stack for Undo Replace:

- Why: LIFO structure (Last IN First Out).
- Advantages: Allows tracking last replacement and reverting it.

3. Implementation Challenges & Solutions

Challenge1: Reducing Dependence on Global Variables

- Problem: As more features were added, using global variables like word_list, sentence_list, and text_data made the code harder to manage and maintain
- Solution: I used a class to group all text data and functions together. This made organized, modular, and easier to expand later.

Challenge2: Adding Undo for Word Replacement

- Problem: There was no way for the user to undo a word replacement if it was done by mistake.
- Solution: I used a stack to store previous versions of the text before replacement. This allowed me to implement an 'Undo' feature, where the last replacement can be reversed.

Challenge 3: Handling Negation in Sentiment Analysis

- Problem: Words like 'not good' can mislead a basic sentiment counter.
- Solution: Implemented a rule to check if 'not' precedes a known sentiment word and inversed the sentiment accordingly.

Challenge 4: Robust User Input Handling

- Problem: Invalid inputs such as incorrect file paths or menu selections.
- Solution: Used loops and exception handling (try-except) to validate inputs and provide friendly prompts.

Challenge 5: Maintaining Trie Consistency After Text Updates

- Problem: The Trie could become outdated after replacing words or loading new text.
- Solution: Rebuild the Trie in functions like replace_word(), undo_last_change(), and display_menu() (option 11) using the latest word list.

4. Testing

Approach:

- ✓ Manual testing with different text inputs (short phrases, long paragraphs, edge cases).
- ✓ Tested individual functions with both valid and invalid data.

Key Test Cases:

- ✓ Sentiment analysis with 'I am not happy' and 'This is not bad.'
- ✓ Autocomplete with prefixes like: 'da', 'fu', 'terr'
- ✓ Edge case: entering a prefix not present in the text(Autocomplete).
- ✓ Next word prediction after common words like: 'I', 'the', 'it'
- ✓ Loading a file with incorrect path (to test error handling).
- ✓ Word cloud frequency generation with and without stop words.
- ✓ Undo replacement: replacing 'bad' → 'good' , then undoing it.

5. Future Work

If extended, the project could include:

- ➢ Machine Learning-Based Sentiment Analysis: Replace lexicon-based approach with trained ML models (e.g., Naive Bayes, BERT).
- ➢ Actual Word Cloud Visualization: Use libraries like wordcloud and matplotlib to generate real word cloud images.

- Graphical User Interface (GUI): Use Tkinter or PyQt to create a user-friendly visual interface.
- Persistent User History: Store user sessions and allow re-analysis of previous texts.
- Multilingual Support: Support for Arabic, French, etc., using appropriate NLP libraries.

6. Conclusion

The Smart Text Analyzer demonstrates practical applications of core data structures in solving real-world language tasks. It highlights how a thoughtful combination of user interface design, efficient structures, and modular code can result in an intuitive and extensible software tool. The project lays the foundation for further development using more advanced NLP techniques and broader feature integration.